

Week 2 Lab / Activity Handout

EDA Design Flow Integration: First RTL Simulation and Waveform Reading

1. Purpose of This Activity

This Week 2 activity gives you a slow, guided first experience with small Verilog RTL modules, simple testbenches, simulation logs, and waveform evidence. The goal is not to build a large circuit. The goal is to become comfortable proving that a very small RTL design behaves as expected before moving to synthesis or implementation flows.

This activity supports the attendance and participation component of the course. It does not directly count toward the 45% FPGA evaluation project or the 45% cell-based IC evaluation project, but it builds the basic skills needed for both.

2. Learning Goals

- Read a small plain Verilog module and identify its ports, wires, regs, and assignments.
- Distinguish combinational behavior from sequential register behavior.
- Run a basic RTL simulation using the provided command pattern or script.
- Inspect a waveform and explain reset behavior, clock edges, and output changes.
- Prepare for Week 3, where the guided FIR filter begins with delay registers and simple arithmetic.

3. Course Scoring Reminder

Component	Weight	Week 2 connection
Evaluation project using the Xilinx FPGA design flow	45%	Week 2 prepares the simulation and waveform skills used later.
Evaluation project using the cell-based digital IC design flow	45%	Week 2 prepares the RTL correctness evidence needed before synthesis.
Attendance and participation	10%	Week 2 activity completion, discussion, and checkpoint questions support this component.

4. Activity Schedule for Today

Time	Activity	Expected Result
0-80 min	Teaching slides: synthesizable RTL, plain Verilog style, testbench structure, reset, and waveform reading	You can explain the basic behavior of small combinational and sequential RTL modules.
80-90 min	Break	Short pause.
90-155 min	Guided lab activity: Parts A-E	You simulate a small combinational module and a small sequential module, then record evidence.
155-165 min	Break	Short pause.
165-180 min	Checkpoint, tool-access notes, and Week 3 preparation	You confirm your simulation evidence and know what to review before the FIR filter begins.

5. Software-Only Scope for Week 2

- Week 2 uses software simulation only.
- No FPGA board execution is required.
- No ASIC tapeout, fabrication, or silicon validation is involved.
- The evidence for this lab is source code, simulation log text, selected waveform signals, and short explanations.

6. Files Used in This Lab

File	Purpose	Student action
mux2.v	Small combinational RTL module	Read the code and predict its output.
tb_mux2.v	Testbench for mux2.v	Run simulation and inspect PASS/FAIL messages.
reg8.v	Small sequential register module	Observe reset and edge-triggered register behavior.
tb_reg8.v	Testbench for reg8.v	Run simulation and inspect clock/reset waveforms.
simulation log	Terminal or simulator transcript	Save the relevant PASS/FAIL lines.
waveform screenshot or saved wave configuration	Visual evidence of signal behavior	Show only the key signals needed to prove behavior.

Note on `timescale`:

Each Verilog source and testbench file includes `timescale 1ns/1ps` at the top. This line tells the simulator the time unit and time precision used during simulation. Do not remove it, because Vivado/XSim may report an error if some files have a timescale and others do not.

7. Part A: Read the RTL Before Running Anything

Before simulation, read the RTL like a design engineer. Identify what enters the module, what leaves the module, and what hardware behavior is implied.

Question	mux2.v	reg8.v
What are the input ports?		
What are the output ports?		
Which signals are combinational?		
Which signals store state?		
Does the module need a clock?		
Does the module need reset?		

8. Part B: Simulate the Combinational Module

The multiplexer chooses one of two 1-bit inputs. When sel is 0, y should follow a. When sel is 1, y should follow b.

```
`timescale 1ns/1ps
module mux2 (
    input wire sel,
    input wire a,
    input wire b,
    output wire y
);

assign y = sel ? b : a;

endmodule
```

Use the instructor-provided script or the following XSim command pattern, adjusted to the server path if needed:

```
xvlog -nolog rtl/mux2.v tb/tb_mux2.v
xelab -nolog tb_mux2 -debug typical -s tb_mux2_sim
xsim -nolog tb_mux2_sim -runall
```

Case	sel	a	b	Expected y	Observed y	Pass?
1	0	0	1	0		
2	1	0	1	1		
3	0	1	0	1		
4	1	1	0	0		

9. Part C: Simulate the Sequential Register Module

The register stores an 8-bit value. The output q should become zero during reset and should update to d only at a rising clock edge after reset is released.

```
`timescale 1ns/1ps
module reg8 (
    input wire      clk,
    input wire      rst_n,
    input wire [7:0] d,
    output reg [7:0] q
);

always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        q <= 8'd0;
    else
        q <= d;
end

endmodule

xvlog -nolog rtl/reg8.v tb/tb_reg8.v
xelab -nolog tb_reg8 -debug typical -s tb_reg8_sim
xsim -nolog tb_reg8_sim -runall
```

Observation	What should happen?	What did you observe?
During reset	q should be 0	
After reset release	q should update only at rising clock edges	
When d changes between clock edges	q should not change immediately	
At the next rising edge	q should capture the current d value	

10. Part D: Waveform Reading Questions

- At what simulation time is reset released?
- Which rising clock edge produces the first normal register update?
- Does q ever change between clock edges? If yes, explain why this would be suspicious.
- Which signal values prove that the multiplexer output is correct?
- Which signal values prove that the register holds its sampled value between rising clock edges?

Answer 1: _____

Answer 2: _____

Answer 3: _____

Answer 4: _____

Answer 5: _____

11. Part E: Evidence Checklist

Evidence item	Required?	Your note
mux2.v and tb_mux2.v	Yes	
reg8.v and tb_reg8.v	Yes	
Simulation log showing PASS/FAIL messages	Yes	
Waveform screenshot for mux2 showing sel, a, b, and y	Yes	
Waveform screenshot for reg8 showing clk, rst_n, d, and q	Yes	
Short explanation of one combinational observation	Yes	
Short explanation of one sequential observation	Yes	

12. Exit Reflection

What is one difference between a Verilog RTL module and a Verilog testbench?

Why should simulation happen before synthesis or implementation?

What should you review before Week 3 begins the FIR filter?

13. Preparation for Week 3

Review module, input, output, wire, reg, assign, always @(*), always @(posedge clk or negedge rst_n), reset, and waveform reading. Week 3 will begin the guided FIR filter using delay registers, simple arithmetic, and testbench-driven verification.